Name: Pradnya Wakode
PRN: 123B1B231
Div: D

Subject: Full Stack Development Lab

Assignment 5

Assignment : Web Application Development using Node.js, Express, and NoSQL

---

1. Introduction

Modern web applications require scalable, efficient, and flexible architectures to handle concurrent users and diverse data types. This project demonstrates the implementation of a full-stack web application using the **Node.js** runtime, the **Express.js** framework, and a **NoSQL MongoDB** database. This stack is widely preferred for its unified use of JavaScript across both client and server environments, enabling rapid development and high performance.

2. Theoretical Background

A. Node.js Runtime

Node.js is a cross-platform, open-source JavaScript runtime environment that executes JavaScript code outside a web browser. It is built on Chrome's V8 engine and utilizes an **event-driven, non-blocking I/O model**. This makes it lightweight and efficient, particularly for data-intensive real-time applications that run across distributed devices.

B. Express.js Framework

Express is a minimal and flexible web application framework for Node.js. It simplifies the server-creation process by providing a robust set of features for web and mobile applications. Key functions include:

- **Routing**: Defining how the application responds to client requests via specific endpoints (URIs).
- **Middleware**: Functions that have access to the request object (req), the response object (res), and the next middleware function in the application's request-response cycle.

C. NoSQL and MongoDB

NoSQL (Not Only SQL) databases are non-tabular and store data differently than relational tables. **MongoDB** is a document-oriented NoSQL database that stores data in **JSON-like BSON documents**.

- **Scalability**: NoSQL databases are designed to scale out by using clusters of machines.
- **Flexibility**: Unlike SQL, MongoDB does not require a predefined schema, allowing the data structure to change over time without downtime.
- **Mongoose**: An Object Data Modeling (ODM) library for MongoDB and Node.js that provides a schema-based solution to model application data, including built-in type casting and validation.

3. Application Architecture

The application follows a standard **Client-Server Architecture**:

1. **Frontend (Client)**: Developed using HTML5, CSS3, and client-side JavaScript. It handles the user interface and sends asynchronous HTTP requests to the server.
2. **Backend (Server)**: The Node.js/Express server acts as the logic layer. It processes requests, performs authentication, and communicates with the database.
3. **Database (Data Layer)**: MongoDB stores persistent data such as user profiles, transaction logs, or service listings.

4. Core Features Implementation

I. RESTful API Development

The communication between the client and server is handled via REST (Representational State Transfer) APIs. These APIs use standard HTTP methods:

- **GET**: Retrieve resources.
- **POST**: Create new resources.
- **PUT/PATCH**: Update existing resources.
- **DELETE**: Remove resources.

II. Security and Authentication

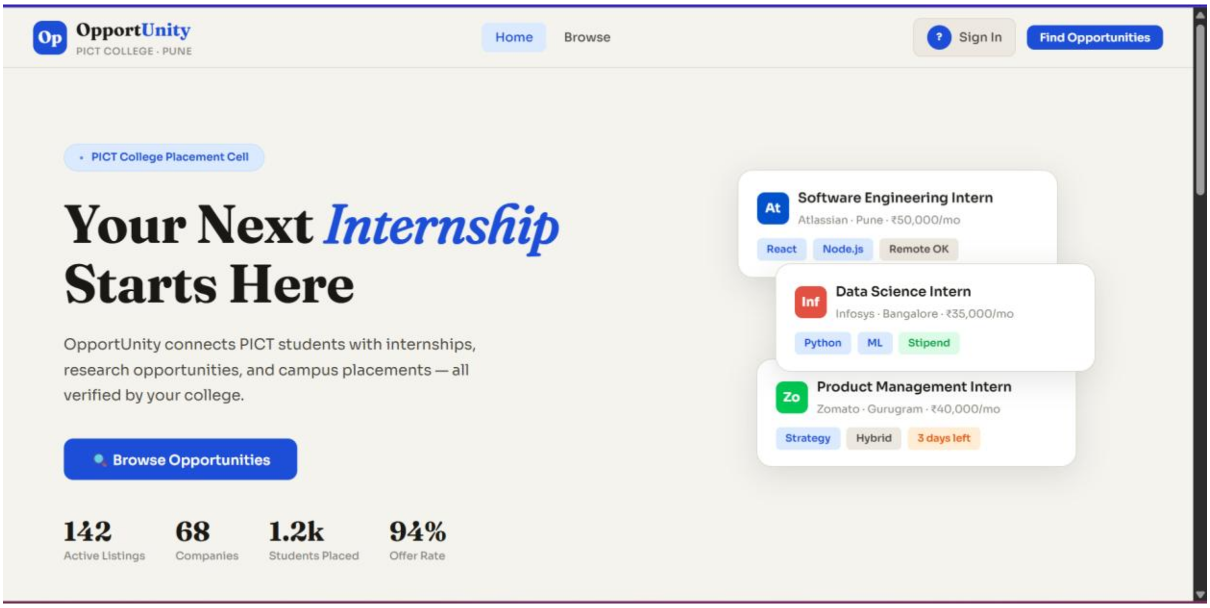
To ensure data integrity and user privacy, the application implements:

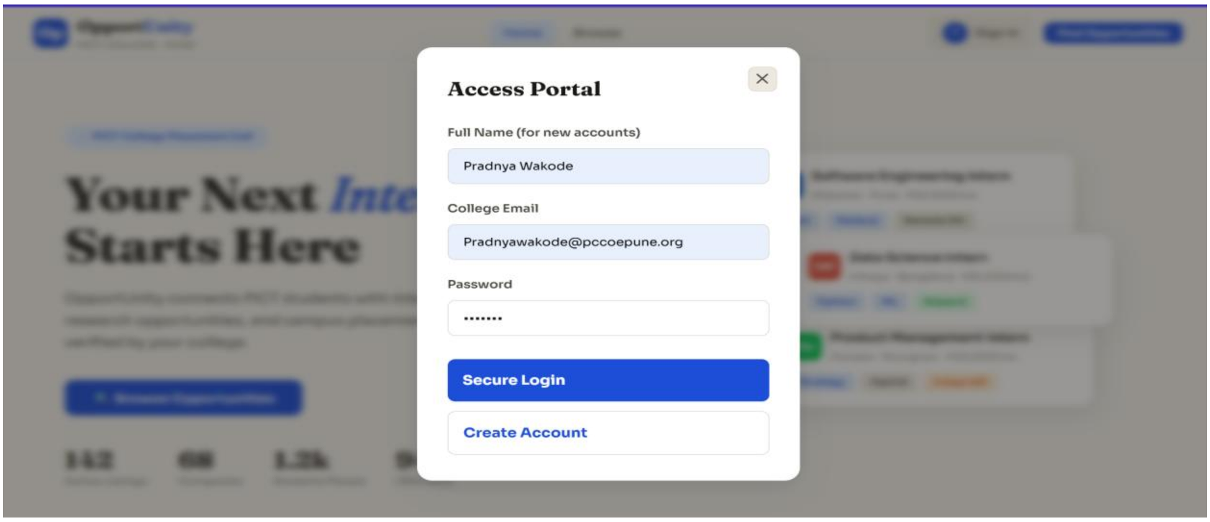
- **Password Hashing**: Utilizing libraries like bcrypt to store mathematical hashes of passwords rather than plain text.
- **Token-Based Authentication**: Using **JSON Web Tokens (JWT)**. Once a user logs in, the server issues a signed token which the client includes in the header of subsequent requests to prove identity.

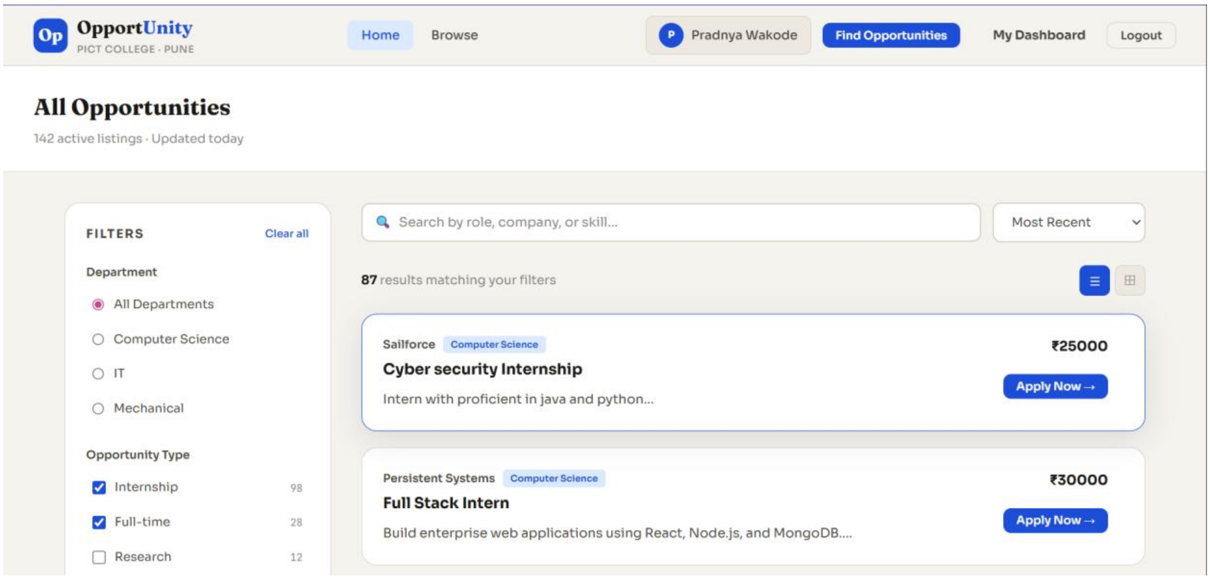
III. Middleware Integration

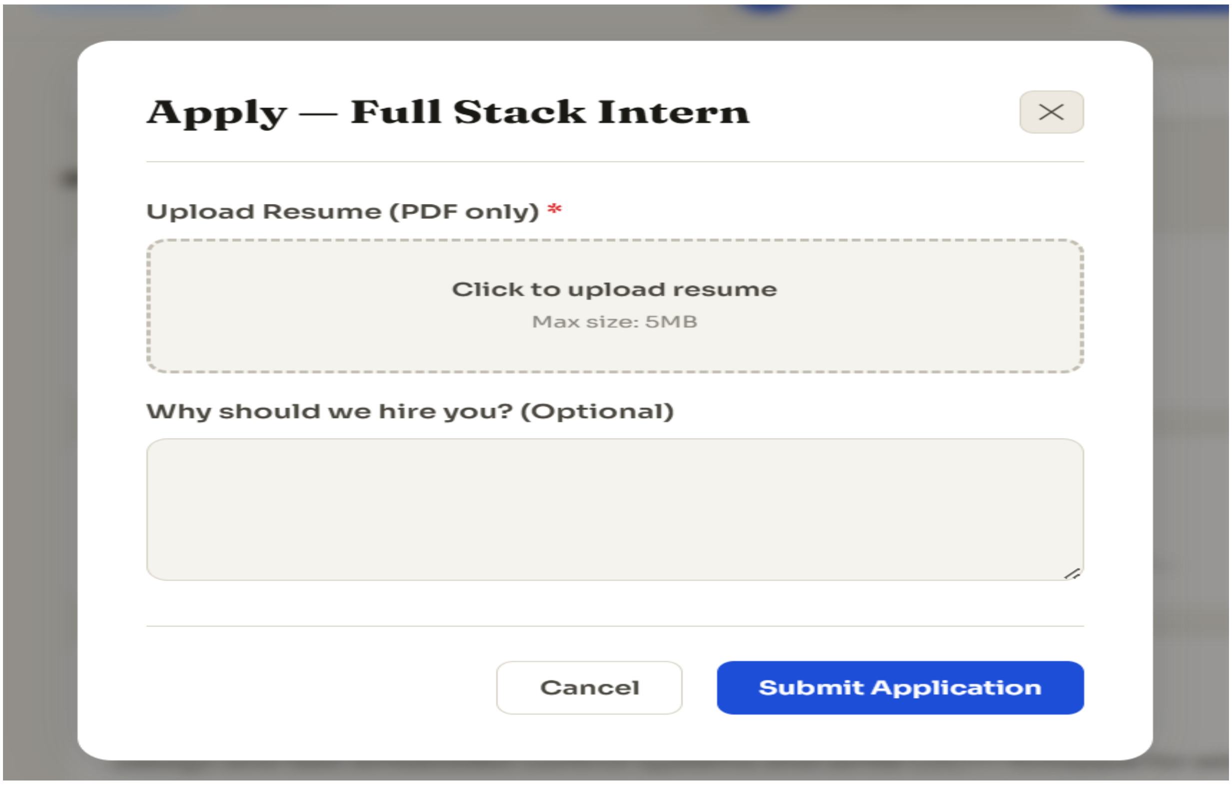
The application utilizes various middleware components:

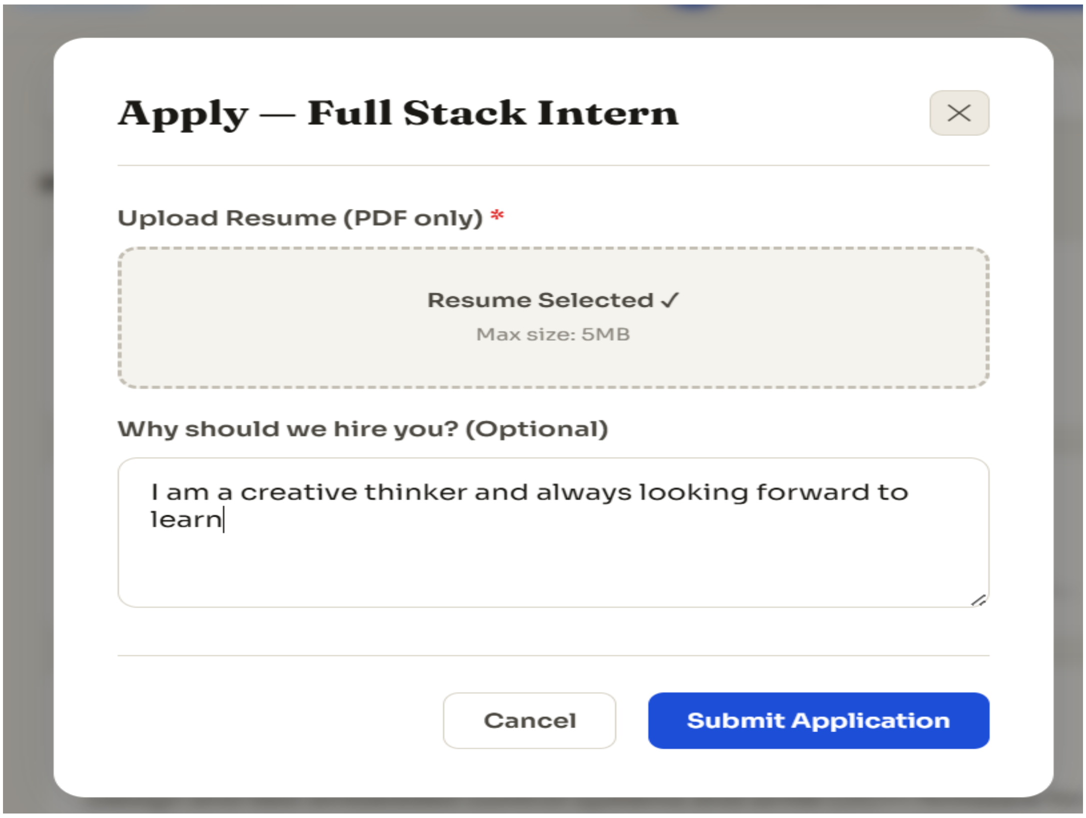
- **CORS (Cross-Origin Resource Sharing)**: Allows the server to accept requests from different domains.
- **Body-Parser**: Parses incoming request bodies in a middleware before your handlers, available under the req.body property.
- **Static Serving**: Automatically serving files like images, CSS, and client-side JS from a designated directory.

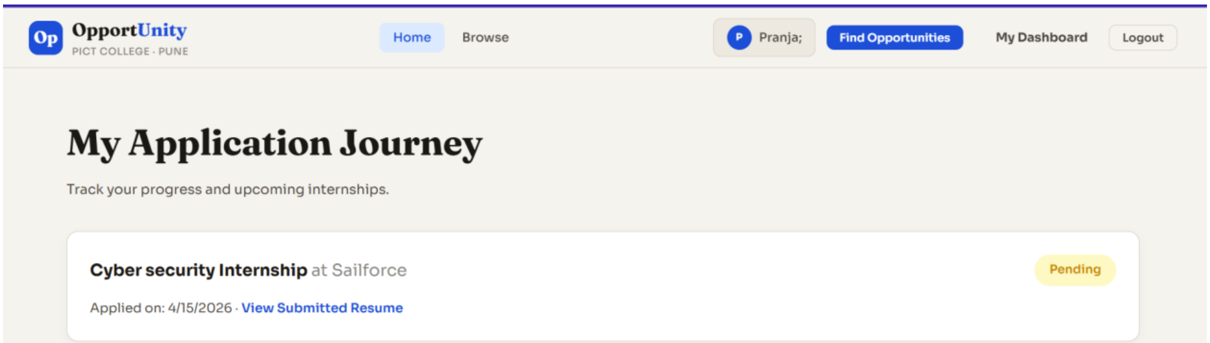





Op
OpportUnity
PICT COLLEGE · PUNE
Home
Browse
P
Pradnya Wakode
Find Opportunities
My Dashboard
Logout
All Opportunities
142 active listings · Updated today
FILTERS
Clear all
Department
All Departments
Computer Science
IT
Mechanical
Opportunity Type
Internship 98
Full-time 28
Research 12
Search by role, company, or skill...
Most Recent
87 results matching your filters
Sailforce
Computer Science
Cyber security Internship
Intern with proficient in java and python...
₹25000
Apply Now →
Persistent Systems
Computer Science
Full Stack Intern
Build enterprise web applications using React, Node.js, and MongoDB....
₹30000
Apply Now →


Apply — Full Stack Intern
Upload Resume (PDF only) *
Click to upload resume
Max size: 5MB
Why should we hire you? (Optional)
Cancel
Submit Application





n \n \n\n

5. Conclusion

By integrating Node.js, Express, and MongoDB, we create a highly responsive and scalable application. The use of NoSQL provides the necessary flexibility to handle evolving data requirements, while the asynchronous nature of Node.js ensures that the server can handle multiple simultaneous connections with minimal latency.